Prose: A Text Formatter

Introduction

Notes on the 6809/FLEX

Historical Notes

Philosophy, Goals, and Capabilities

Basic Units of Text

A General Look at Directives

Controlling the Formatting Environment

Short Directive Table

Details on Directives

Running Prose on the 6809/FLEX

Introduction

Preparation and editing of computer-oriented documentation is tedious. This process can be made easier by computerized text-processing tools such as text editors and formatters. This guide describes a text-formatting program named Prose and the way in which it is invoked on the 6809/FLEX series of computers. Prose and this instruction guide are oriented toward the preparation of computer-oriented documentation; so this guide assumes basic knowledge of computer-related text processing tools.

The contents of this guide are adapted from the Prose Instruction Manual written by John P. Strait (copyright 1978, University Computer Center, University of Minnesota), as described in Pascal News (No. 15) in September 1979.

Notes for the 6809/FLEX

This document describes the version of Prose installed on the 6809/FLEX . As a result, the default form of Prose commands have been made to correspond to 6809/FLEX usage, and changes have been made in the output devices to fit those available on the 6809/FLEX. Specific machine-dependent characteristics of this version of Prose are are flagged by "(6809/FLEX)". Several items in the original Prose manual referring to the idiosyncracies of another machine have been shamelessly removed.

****

Text examples in this manual have been extracted from Alice's Adventures In Wonderland by Lewis Carroll.

****

Historical Notes

Most text formatting programs available today descend from one of several original programs. Among these is RUNOFF, developed on the Dartmouth Time-Sharing System in the 1960s. Later, the Call-a-Computer system provided a RUNOFF version called EDIT RUNOFF as a text editor command. In 1972, Michael Huck, working on the University of Minnesota's MERITSS system (a CDC 6400 running the KRONOS operating system), began to develop a version of EDIT RUNOFF that he called TYPESET. TYPESET went through many developmental changes, stablizing somewhat in early 1977 at version 5.0, which is written in CDC COMPASS assembly language. Prose, written in Pascal, was developed over a year's time starting in the spring of 1977. The design was influenced heavily by TYPESET, making Prose one of the many descendants of RUNOFF.

Prose, with minor changes, was installed on the Univac 1100 series computers in early 1980 by Michael S. Ball of the Naval Ocean Systems Center. He converted this version from the Univac to the PDP-11 in July 1980 at Oregon Software. The conversion to 6809/FLEX was done by Luc E. Boullart at the State University, Ghent Belgium.

Philosophy, Goals, and Capabilities

Prose is intended primarily for the preparation of machine-retrievable documentation, and this has influenced the choice of its repertoire of abilities. TYPESET was intended as a "versatile text information processor commonly used to typeset theme papers, term papers, essays, letters, reports, external documentation . . . , and almost any other typewritten text" [Typeset 5.0 Information, Copyright 1977 by Michael Huck]. In spite of these aspirations, no program can be all things to all people, and so it

is with Prose. It is intended that Prose be able to do most of the things needed to produce high-quality computerized text.

The design of Prose had several goals, among them, that high-quality results should be produced with a minimum number of directives and that Prose should have about 90% of the abilities that you think are useful and that the 10% it doesn't have should be so esoteric as to be non-essential. Some text formatters take the approach of providing a minimum set of built-in abilities, along with a "general and powerful" feature such as macros. The idea is that you can accomplish anything you want (no matter how much effort it will take) by defining appropriate macros. The problem with this approach is that the user is forced to learn a complicated feature to produce any but the most trivial results.

The philosophy behind Prose is that the user should not be overwhelmed by a large number of complicated directives; that the syntax of the directives should be consistent; that the text, not the directives, should stand out. Because of this goal of simplicity, Prose may or may not be the tool for a given application. The following two tables should aid you in deciding whether to use Prose.

Prose . . .

a. Prose has a small number of commands providing an easily learned set of basic formatting abilities.
b. Prose can do underlining and discretionary hyphenation.
c. Prose can remember and restore the text processing environment.
d. Prose can produce mixed-case or upper-case-only output from either mixed-case or upper-case-only input.
e. Prose can accumulate and produce a sorted index, referring to page numbers.
f. Prose can print selected pages on request.
g. Prose can format text in pages with headers, footers, and other aids.
h. Prose can fill and justify text to specified margins.
i. Prose is a portable program, written in standard Pascal, using ASCII as its internal character code. Prose is written to encourage its transportation between computers with different hardware and different operating systems.

. . . and Cons

a. Prose cannot control photo-typsetting machines.
b. Prose cannot do graphics.
c. Prose does not have multi-column ability.

d. Prose does not have macros, variables, or other programming language-like features.
d. Except for its indexing ability, Prose cannot store text and retrieve it later.
e. Prose does not have tabs.
f. Prose does not have directives to do everything you always wanted to do.

Basic Units of Text

Some of the basic units of natural language are the word, the phrase, the sentence, and the paragraph. In text formatting, the basic units are the word, the line, and the paragraph. A word is defined as any non-blank string of characters, with a blank on either side. For the purposes of formatting, a punctuation character is part of the word it is next to. By default, Prose reformats its input by filling words into lines, adding blanks to justify the lines to left and right margins, and printing lines together to make paragraphs. In filling lines, Prose does not pay any attention to the original positions of the words, but instead fills as many words as possible into the output lines, preserving the original order. The following example illustrates this process of filling and justifying.

Input to Prose:

"When we were little," the Mock Turtle went on at last, more calmly, though still sobbing a little now and then, "We went to school in the sea. The master was an old Turtle--we used to call him Tortoise--"
"Why did you call him Tortoise, if he wasn't one?" Alice asked.
"We called him Tortoise because he taught us," said the Mock Turtle angrily. "Really you are very dull!"
"You ought to be ashamed of yourself for asking such a simple question," added the Gryphon; and then
they both sat silent and looked at Alice, who felt ready to sink into the earth.

Output from Prose:

"When we were little," the Mock Turtle went on at last, more calmly, though still sobbing a little now and then, "we went to school in the sea. The master was an old Turtle--we used to call him Tortoise--"
"Why did you call him Tortoise, if he wasn't one?" Alice asked.
"We called him Tortoise because he taught us," said the Mock Turtle angrily. "Really you are very dull!"
"You ought to be ashamed of yourself for asking such a simple question," added the Gryphon; and then they both

sat silent and looked at Alice, who felt ready to sink into the earth.

Most of text formatting is filling and justifying. In the absence of special instructions (called directives), Prose will fill all of the input words into output lines and justify all of those lines.

The distinction between one paragraph and the next is defined by a justification break, which causes Prose to stop filling the current output line and print it without justifying. Since the break is one of the most frequently used instructions (as well as one of the simplest), it can be indicated in many ways. Paragraphs can be separated (broken) by one or more blank lines, by leading blanks typed on an input line (a paragraph indentation), or by the Prose ".BREAK" directive. The following example demonstrates these three methods.

Input to Prose

```
At last the Gryphon said to the Mock turtle "Drive on,
old fellow!  Don't be all day about it!" and he went
on in these words:--

"Yes, we went to school in the sea, though you mayn't
believe it--"
.BREAK
"I never said I didn't!" interrupted Alice.
.BREAK
"You did." said the Mock Turtle.
     "Hold your tongue!" added the Gryphon, before Alice
could speak again.
```

Output from Prose:

```
At last the Gryphon said to the Mock Turtle "Drive on old
fellow!  Don't be all day about it!" and he went on in
these words:--

"Yes, we went to school in the sea, though you mayn't
believe it--"
"I never said I didn't!" interrupted Alice.
"You did." said the Mock Turtle.
     "Hold your tongue!" added the Gryphon, before Alice
could speak again.
```

When you use one of these methods to create a paragraph, Prose only does a justification break. Prose will not skip lines or indent unless blank lines or indentations explicitly appear in the input file. You can do fancier things with the ".PARAGRAPH" directive, to be introduced later.

A General Look at Directives

In its default mode, Prose automatically fills and justifies output lines, formatting the output in pages. Directives instruct Prose to do anything fancier. Directives can change the margins, control options, and define the type of output device you intend to use.

A line of directives is indicated by the directive escape character in the first column of an input line. The period is the default directive escape character (although you can change it if you wish) because it seems unlikely that anyone will want to type a period in the first column of a line of text. Several directives can be typed on the same line, provided that they are separated by the directive escape character. For example:

```
.BREAK.SKIP 2.MARGIN( L5 R65 )
```

Some directives take the remainder of the line as their parameter, so no other directives can follow these. Long directives may extend to several lines. Continuation lines are indicated by a plus sign (+) typed in column one. The continuation may be made anywhere that a blank is allowed. For example:

```
.FORM( [ /// L58 // #73 'PAGE' p /// ]
+      [ /// L58 //     'PAGE' p /// ] )
```

Examples in this guide will show directives typed in upper case, but both upper-case and lower-case characters may be mixed.

Every directive begins with the name of the command, "MARGIN" for instance. The name can be abbreviated to three letters (in fact, Prose only examines the first three letters.) The directive name may be followed by a parameter. In the absence of a parameter, default values are used. The parameter has four forms:

1) The absence of any parameter.
2) A single numeric value.
3) The remainder of the directive line.
4) A information enclosed in parentheses, consisting of descriptors defined by the directive itself.

When a numeric value is required (for a parameter or as part of a descriptor), an explicit positive integer may be given. In many directives, a relative value may be used. A relative value, specified by a plus or minus sign before the integer, indicates that the old value should be increased or decreased by the number of the integer. In the following example, the left margin is set to 10 and the right margin to 70. Then, the margins are squeezed together by 5 characters on both sides.

```
.MARGIN( L10 R70 )
.MARGIN( L+5 R-5 )
```

Controlling the Formatting Environment

The formatting environment is defined to be all the options and specifications that direct Prose as it produces formatted output from unformatted input. The concepts that make up the formatting environment can be loosely grouped into six areas; directives control each one:

1) INPUT controls the meaning and treatment of characters on the input file.
2) OUTPUT describes the type of output device for which the formatted result is intended.
3) FORM specifies the format of the page into which the running text will be inserted. This includes information on the placement of titles, footers, and the like.
4) MARGIN sets the left and right margins.
5) PARAGRAPH describes special actions for the beginning of each paragraph.
6) OPTION controls the rest of the miscellaneous options that affect the text formatting process.

Of these six groups, the INPUT, MARGIN, OPTION, and PARAGRAPH settings are likely to be changed often throughout the test. The number of different settings, however, will probably be small. To accomodate these needs, a simple device is available for these four directives.

The setting of options controlled by these directives follows this syntax:

```
.directivename( parameters )
```

where the parameters consist of a key letter followed by option settings. For instance:

```
.MARGIN( L5 R60 )
```

sets the left margin to 5 and the right to 60. Each time one of the four directives is processed, Prose saves the new values in a keep buffer. Ten keep buffers (numbered 0 through 9) are associated with each directive. A keep parameter may be used to specify the buffer to be used; if not specified, the values are saved in the numerically next buffer.

Old values may be recalled by using the following form:

```
.directivename number
```

For example:

```
.MARGIN 5
```

sets the margins to the values stored in keep buffer 5.

If no parameter is specified, the values are set to those stored in the numerically previous keep buffer. Since the keep number is automatically incremented when the parenthesis form is used and automatically decremented when no parameter is given, the keep buffers can be used as a stack.

```
.MARGIN( L0 R70 )

   ...

.MARGIN( L10 R60 )

   ...

.MARGIN
```

In the previous example, the last MARGIN directive resets the margins to their previous values: left 0 and right 70.

Short Directive Table

Directive	Meaning (action)	Break	Parameter type
BREAK	break justification	*	-none-
COMMENT	no action		remainder of line
COUNT	set page count		numeric
FORM	define page format	*	(...)
INDENT	indent following line	*	numeric
INPUT	set input parameters	*	(...) or numeric
INX	store index entry		remainder of line
LITERAL	print literal text		remainder of line
MARGIN	set margins	*	(...) or numeric
OPTION	set options	*	(...) or numeric
OUTPUT	set output parameters		(...)
PAGE	eject to top of page	*	numeric
PARAGRAPH	set paragraphing params		(...) or numeric
RESET	reset directive defaults	*	(...)
SELECT	select pages to print	*	(...)
SKIP	skip output lines	*	numeric
SORTINDEX	sort and print index	*	(...)
SUBTITLE	set the subtitle		remainder of line
TITLE	set the main title		remainder of line
UNDENT	undent following line	*	numeric
WEOS	write end of section	*	-none-

The directives marked with an asterisk (*) cause a justification break before they are processed, since they affect the filling and justifying environment.

(...) indicates that the parameter is enclosed in parentheses and is described in detail along with the description of the directive itself.

Details on Directives

BREAK

Causes a justification break

COMMENT

Prose treats the remainder of the directive line as a comment; i.e., it is ignored. The COMMENT directive allows you to include information in the source of your document that will not be printed in the formatted copy.

COUNT number

COUNT

Sets the page counter. The numeric parameter can be relative; for example, ".COUNT +1" increments the page number by one. Without a parameter, the default sets the page number to one.

FORM (parameters)

FORM

Defines the page format, including titles, footers, date/time, and the top and bottom of the page. The argument consists of parameters, followed by an optional field width, if appropriate. For example, "T:30" prints the title in a field of 30 characters. Text lines are built by the FORM directive from left to right, starting in the first printable column, although the tabbing specification may be used to alter that. The following table describes the available FORM specifications.

Key Char	Meaning	Default	Field Width
C	24 hour clock as hh.mm.ss	(15.37.58)	8
D	raw date as yy/mm/dd	(78/02/13)	8
E	nice date as dd Mmm yy	(13 Feb 78)	9
Ln	fill in n lines of running text		
Pf	current page number, f selects the form		3
	N or n arabic numerals (default)		[the field
	L upper case letter		width will
	l lower case letter		be expanded
	R upper case roman numerals		if needed]
	r lower case roman numerals		
S	subtitle		its length
T	main title		its length
W	wall clock as hh:mm AM	(3:37 PM)	8
	or hh:mm PM		
#n	tab forward or backward to absolute column n		
'...'	print literal text		
/	print an end of line (by itself, a blank line)		
/n	print -n- ends of lines		
[	define top of page		
]	define bottom of page		

default form (6809/FLEX)

```
.FORM( [ // T #62 E /// L54 /// #33 '- ' PN:1 ' -' //// ] )
```

The FORM directive is processed interpretively. The FORM argument is re-scanned as each page of output is produced, so that a change in a title buffer with the TITLE or SUBTITLE directive will change the title or subtitle on the next page.

The top of page definition is used for several things. By using the OUTPUT directive, you can request Prose to send a page eject to the output device when it reaches the top of a page. You also can request Prose to pause at the top of each page to allow you to change paper. At the end of the document, Prose does one last page eject and continues to interpret the FORM specification until

it reaches the top of page. (This ensures that any commands specified for the bottom of the last page -- such as a page number -- are executed.)

Prose increments the page number at the bottom of the page specification, so if you print the page number both before and after the bottom of page definition you will get different numbers.

Once you understand the FORM directive, you can easily produce fancy page formats. You can, for example, design a FORM that will print the page number at the right of odd numbered pages and at the left of even pages. This is done with a FORM that defines two pages with two "["s and two "]"s:

```
.FORM ( [ // T #62 E /// L56 // #63 'PAGE' P /// ]
+       [ // T #62 E /// L56 //     'PAGE' P /// ] )
```

In the absence of any of the parameters described above, no special page formatting is done. The result is a FORM consisting of a single L specification defining an infinite number of lines per page. In this mode, a PAGE directive with no parameter will put 5 blank lines between sections of text.

INDENT number

INDENT

Indents the following line by a certain number of spaces. In the absence of a parameter, the default is 5.

INPUT (parameters)

INPUT number

INPUT

The INPUT directive is used to define the input environment, that is, the interpretation of characters in the input file. The parameters, which can be given in any order, consist of a key letter followed by a value. The following table summarizes the parameters.

Key Letter	Meaning	Type	Default	Relative
B	explicit blank character	character	nul	n/a
C	case-shift character	character	nul	n/a
D	directive escape character	character	.	n/a
H	hyphenation character	character	nul	n/a
K	keep	number	next	no
U	underline character	character	nul	n/a
W	input width	number	150	no

The default value is assigned when Prose begins processing. The default value also will be set if the key letter is given by itself. The value of a parameter changes only when a is given.

B: The explicit blank character indicates a blank that Prose should not tamper with. Thus, if the cross-hatch (#) is specified as the explicit blank:

```
.INPUT( B# )
```

then two words that are separated by an explicit blank:

```
Mr. Smith
```

will never be split from one line to the next, and Prose will never fill blanks in between the words to justify a line.

C: The case-shift character must be used to create mixed-case output from upper-case-only input. When a case-shift character is specified, Prose automatically shifts all upper-case letters to lower case. To specify an upper-case letter, you must surround letters with the case-shift characters, causing a shift-up and shift-down. Since most upper-case letters are at the beginning of a word (following a blank), another method, called stuttering, is to double the first character of a word to achieve upper-case characters. The following example demonstrates the production of mixed-case output from upper-case-only input.

Input to Prose:

```
.INPUT(C^)
TTHE MMOCK TTURTLE WENT ON.
     "^W^E HAD THE BEST OF EDUCATIONS--IN FACT, WE WENT TO
SCHOOL EVERY DAY--"
     ^I'VE^ BEEN TO A DAY-SCHOOL, TOO," SAID AALICE.
"^Y^OU NEEDN'T BE SO PROUD AS ALL THAT."
     "^W^ITH EXTRAS?" ASKED THE MMOCK TTURTLE, A
LITTLE ANXIOUSLY.
     "^Y^ES SAID AALICE: "WE LEARNED FFRENCH AND MUSIC."
     "^A^ND WASHING?" SAID THE MMOCK TTURTLE.
     "^C^ERTAINLY NOT," SAID AALICE, INDIGNANTLY.
     "^A^H TTHEN YOURS WASN'T A REALLY GOOD SCHOOL," SAID
THE MMOCK TTURTLE IN A TONE OF GREAT RELIEF.  "^N^OW,
AT ^OURS^, THEY HAD, AT THE END OF THE BILL, '^F^RENCH,
MUSIC, ^AND WASHING--^ EXTRA.'"
```

Output from Prose:

```
The Mock Turtle went on.
     "We had the best of educations--in fact, we went to
school every day--"
     "I'VE been to a day-school, too," said Alice.  "You
needn't be so proud as all that."
     "With extras?" asked the Mock Turtle, a little
anxiously.
     "Yes," said Alice: "we learned French and music."
```

"And washing?" said the Mock Turtle.
"Certainly not," said Alice, indignantly.
"Ah Then yours wasn't a really good school," said the Mock Turtle in a tone of great relief. "Now at OURS, they had, at the end of the bill, "French, music, AND WASHING--extra.'"

At first glance, the stuttering method may seem clumsy, but experience shows that it is reasonably easy to get used to. To enter words that already have a double letter at the beginning (like llama and oops), merely precede the word with two case-shift characters, causing a shift-up/shift-down (^^LLAMA and ^^OOPS). The case-shift character does not need to be used unless you want to create mixed-case output from upper-case-only input. If possible, use mixed-case input to create mixed-case output.

D: The directive escape character is the character you type in the first column of an input line to flag it as a directive line.

H: The hyphenation character is used to define hyphenation points within words. Sometimes a long word will cause many blanks to be inserted to justify the preceding line. Prose will hyphen-ate such a word if you have defined the syllable boundaries within it. Of course, not all the syllable boundares need be specified, only those where you want Prose to be able to split a word. For example, if the hyphenation character is set to be the slash (/), you might type "syncopation" as "syn/co/pa/tion". Prose will insert a hyphen (-) only when the characters on both sides of the hyphenation point are letters. You might type "hyper-active" as "hyper-/active", and Prose will split the word if necessary, without adding a superfluous hyphen. If Prose is forced to insert more blanks than a certain threshold (set with the OPTION directive), it will issue a message suggesting that you insert hyphenation characters.

K: The keep parameter explicitly specifies the keep buffer to be used to store the new input options. The default is for Prose to use the numerically next buffer.

U: Text surrounded by the underline character will be underlined. Blanks are not underlined, but explicit blanks are.

W: The input width is used to specify the number of characters to be read from each input line. If your input lines have sequencing information at the right of each line, you will need to set the width to an appropriate value.

INX text

Enters the remainder of the line together with the current page number as an index entry. If the formatted text migrates from page to page, the resulting index will always be correct.

LITERAL text

Prints the remainder of the line on the output file. The special processing for upper/lower case, underlining, and literal blanks is performed on the text of the parameter, and then it is printed as a single output line. This output line is printed independently of filling and justifying and page formatting processes; it is transparent to the usual Prose formatting and is not counted as an output line. The LITERAL directive is useful for producing special printer control characters on some systems. LITERAL is of little use on the 6809/FLEX and is described for the sake of completeness.

MARGIN (parameters)

MARGIN number

MARGIN
The MARGIN directive sets the left and right margins for filling and justifying. The left margin is the number of leading spaces before the first printed character, and the right margin is the column number of the last printed character. Thus, subtraction of the left margin from the right margin gives the number of printed columns. The parameters, which may be given in any order, consist of a key letter followed by a value. The following table lists the parameters.

Key Letter	Meaning	Type	Default	Relative
K	keep	number	next	no
L	left margin	number	0	yes
R	right margin	number	70	yes

Prose assigns the default value when it begins processing. The default value also is set when the key letter is given by itself. A parameter value is changed only when a specification is given.

The keep parameter explicitly specifies the keep buffer to be used to store the new margins. The default is for Prose to use the numerically next buffer.

OPTION (parameters)

OPTION number

OPTION
Miscellaneous options that affect the text formatting process are gathered together in the OPTION directive. These options are summarized in the following table. For switch options, "+" is on and '-' is off.

Key Letter	Meaning	Type	Default	Relative
E	print error messages	switch	+	n/a
F	fill output lines	switch	+	n/a
J	justification limit	numeric	3	no
K	keep	numeric	next	no
L	left justify	switch	+	n/a
M	multiple blanks	switch	+	n/a
P	2 blanks after periods	switch	+	n/a
R	right justify	switch	+	n/a
S	spacing	numeric	1	no
U	shift to upper case	switch	-	n/a

Prose assigns the default value when it begins processing. The default value also is set when the key letter is given by itself. A parameter value is changed only when a specification is given.

E: Error messages appear in the formatted text of the main output files at the approximate location of the errors. Error messages are suppressed when the E option is set to "E-".

F: Output lines are automatically filled and justified as described in the section "Basic Units of Text". If the fill switch is turned off, Prose will print the input lines as they are, without reformatting to fill up the output lines. In effect, a justification break is done after each input line.

J: In justifying the left and right margins of an output line, Prose has to insert blanks that are not explicitly on the input file. The justification limit controls the point at which Prose will attempt to hyphenate a word. If, for instance, the justification limit is three, then the hyphenation process will be invoked when Prose has to insert three blanks between any adjacent words on a line. If hyphenation is not possible, or Prose is not able to bring the number of inserted blanks below the limit, an error message is printed.

K: The keep parameter explicitly specifies the keep buffer to be used to store the new options. The default is for Prose to use the numerically next buffer.

L:
R: The left and right justify switches work together to determine the kind of justification to be done. If both switches are on, output lines are justified to both the left and right margins. If both switches are off, the lines are centered between the two margins. If one is on and one is off, the result is one straight margin (either left or right) and one ragged margin. The following demonstrates these four options.

```
.OPTION( L+ R+ ) :
```

"You couldn't have wanted it much," said Alice; "living at the bottom of the sea."

"I couldn't afford to learn it," said the Mock Turtle with a sigh. "I only took the regular course."

"What was that?" inquired Alice.

"Reeling and Writhing, of course, to begin with," the Mock Turtle replied; "and then the different branches of Arithmetic--Ambition, Distraction, Uglification, and Derision."

"I never heard of 'Uglification,'" Alice ventured to say. "What is it?"

The Gryphon lifted up both its paws in suprise. "Never heard of uglifying!" it exclaimed. "You know what to beautify is, I suppose?"

.OPTION(L- R-) :

"Yes," said Alice doubtfully: "it means--to--make--anything--prettier."

"Well, then," the Gryphon went on, "if you don't know what to uglify is, you are a simpleton."

Alice did not feel encouraged to ask any more questions about it: so she turned to the Mock Turtle, and said "What else had you to learn?"

"Well, there was Mystery," the Mock Turtle replied, counting off the subjects on his flappers--"Mystery, ancient and modern, with Seaography: then Drawling--the Drawling-master was an old conger-eel, that used to come once a week: he taught us Drawling, Stretch-
ing, and Fainting in Coils."

.OPTION(L+ R-) :

"What was that like?" said Alice.

"Well, I can't show it to you, myself," the Mock Turtle said. "I'm too stiff. And the Gryphon never learnt it."

"Hadn't time," said the Gryphon: "I went to the Classical master, though. He was an old crab, he was."

"I never went to him," the Mock Turtle said with a sigh. "He taught Laughing and Grief, they used to say."

"So he did, so he did," said the Gryphon, sighing in turn; and both creatures hid their faces in their paws.

"And how many hours did you do lessons?" said Alice, in a hurry to change the subject.

.OPTION(L- R+)

"Ten hours the first day," said the Mock Turtle: "nine the next, and so on."

"What a curious plan!" exclaimed Alice.

"That's the reason they're called lessons," the Gryphon remarked: "because they lesson from day to day."

This was quite a new idea to Alice, and she thought it over a little before she made her next remark. "Then the eleventh day must have been a holiday?"

"Of course it was," said the Mock Turtle.

"And how did you manage on the twelfth?" Alice went on eagerly.

"That's enough about lessons," The Gryphon interrupted in a very decided tone. "Tell her something about the games now."

M: If the multiple blanks switch is on, multiple blanks in the input file are considered to be significant. That is, if there are several blanks between two words in the input file, there will be at least that many in the output file. Prose may add more blanks during justification. If the switch is off, multiple blanks will be changed into a single blank.

P: The 2 blanks after periods option will ensure that each period already followed by at least one blank will be followed by at least two blanks. Prose will not add blanks before justifying if there are already two. This makes for nice spacing in your final copy even if you are not careful about typing spaces in the original.

S: Set the spacing option to 2 or 3 to get double-spaced or triple-spaced output, respectively. (Default is 1.)

U: Since some output devices are not able to handle mixed-case files, you can shift all lower-case letters to upper-case letters by selecting the shift to upper case option. This option is also handy for printing an entire unit, such as a sample program, all in upper case.

OUTPUT (terminal-type parameters)

The OUTPUT directive defines important aspects of the output device to which you will send the formatted text. The OUTPUT directive may be used only once. It must appear in one of only two places: before any lines are printed on the output device or immediately after the directive ".RESET(OUTPUT)".

Terminal-type may be one of the following.

(6809/FLEX) All of the following is specific to the 6809/FLEX.

ASC ASCII terminal, uses back space for underlining, but is otherwise the same as LPT below. There is, however, a difference in the handling of pauses for page eject. (See the P option below.)

LPT Line printer, using overprinting with a carriage return to do underlining. This is the default output device.

VTR Video Terminal (DEC VT100 or VT52). This does underlining by means of the control sequences defined for these terminals.

FAC FACIT 4510 and 4512 type printers. This does the undelining by means of the appropriate control sequences.

The parameters define further characteristics of the output device, along with several global output options. The parameters, which may be given in any order, are listed in the following table.

Key Letter	Meaning	Type	Default
E	page eject at top of page ("[" in FORM description)	switch	-
P	pause at top of page	switch	-
S	shift output lines to the right	numeric	0
U	underlining is available	switch	+

E: The page eject option will print a form feed each time a "[" is encountered in the FORM specification.

P: Selection of the pause option will cause Prose to stop printing and await operator acknowledgement each time a "[" is encountered in the FORM specification. On an ASC terminal, Prose will sound the bell and wait for a carriage return to be entered.

For an LPT output device, no action will be taken.

S: All output that Prose produces can be shifted to the right by any number of spaces up to 50. This makes it easy to center the output on a wide printer page.

U: If the destination terminal does not have underlining ability and your input does underlining, the underlining available option should be turned off to prevent Prose from trying to generate overprinted underlines.

PAGE number

PAGE
Causes a page eject if there are fewer than the specified number of lines remaining on the current page. If no parameter is given, PAGE does an unconditional page eject.

PARAGRAPH (parameters)

PARAGRAPH number

PARAGRAPH
Paragraphs can be indicated by any of the methods introduced in the section "Basic Units of Text". The PARAGRAPH directive provides a more versatile method of creating paragraphs.

The PARAGRAPH directive specifies the action to be taken when a new paragraph is signaled by the paragraph flag character in the first column of an input line. An automatic indent or undent can be selected, an automatic skip and/or automatic page eject can be specified, and you can even have Prose automatically number the paragraphs.

Key	Letter	Meaning	Type	Default	Relative
	F	paragraph character	character	nul	n/a
	I	Automatic indent	number	0	no
	K	keep	number	next	no
	N	number generator		none	n/a
	P	automatic page eject	number	0	no
	S	automatic skip	number	0	no
	U	automatic undent	number	0	no

Prose assigns the default value when it begins processing. The default value also is set when the key letter is given by itself. A parameter value is changed only when a specification is given.

F: The paragraph flag character invokes this collection of paragraphing actions when it appears in the first column of an input line. Note that this character must be set in at least one PARAGRAPH directive, or none of these actions will work.

I:
U: The automatic indent or automatic undent is applied to the first line of the paragraph (see the description of INDENT and UNDENT). If the number generator is used, the indent or undent is applied after the number is generated.

N: If the number generator is specified, a new number (or letter) will be generated for each occurrence of the paragraph flag character. The number generator is initialized to 1 each time new paragraph settings go into effect. Resumption of an old setting will also resume the old numbering. The number replaces the paragraph flag character when the line is formatted. The number generator parameter has the form: Nfn.

f selects the numeric form:

-blank-	no numbering
N or n	arabic numerals
L	upper-case letter
l	lower-case letter
R	upper-case roman
r	lower-case roman

n is the field width, which will be expanded if needed. If, for example, you want an arabic numeral with three spaces left for the numeral, the command is 'Nn3'.

P: The automatic page eject simulates the effect of the directive

.PAGE number

before the first line of the paragraph. If this parameter is set to 4, for instance, a page eject is done if fewer than four lines will be left at the bottom of the page for the paragraph. The command is applied after the automatic skip.

S: The automatic skip is done before the first line of the paragraph, and functions the same as a skip directive.

K: The keep parameter explicitly specifies the keep buffer to be used to store the new paragraph options. The default is for Prose to use the numerically next buffer.

RESET

RESET (parameters)

RESET (EXCEPT parameters)

The RESET directive sets directives to their default values. If you have changed the values of many directives (such as FORM, MARGIN, or OPTION), the simple command

```
.RESET
```

resets the values of all directives to their defaults. You also may reset directives selectively by using the second form of the command. For example,

```
.RESET( MARGIN OPTION )
```

only resets the MARGIN and OPTION directives. Directives also may be excluded selectively. For example,

```
.RESET( EXCEPT FORM OUTPUT )
```

resets all directives with the exception of FORM and OUTPUT.

Any of the following parameters may be reset:

COUNT	FORM	INPUT	INX
MARGIN	OPTION	OUTPUT	PAGE
PARAGRAPH	SELECT	SUBTITLE	TITLE

The values of parameters for most directives are set to their defaults (listed with the description of each directive), except for the keep parameters, which are set to "K0". For the COUNT, INX, and PAGE directives, however, the action is different. A reset of COUNT sets the page counter to 1; a reset of INX deletes all index entries that have accumulated; and a reset of PAGE causes a page eject. In addition, a FORM or OUTPUT reset also causes a page eject, since these directly affect the printed result.

SELECT (parameters)

You may use the SELECT directive when you need to print only certain pages of a document. The entire text will be formatted, but only the selected pages will be printed. Processing time will not be reduced much, but printing time will be. The descriptor consists of page numbers separated by spaces. Two page numbers separated by a colon (:) will select the span of pages between the pages, inclusive. The second page number may be specified relative to the first. The following example selects pages 3, 5, 10 through 15, and 20 through 25 to be printed.

```
.SELECT( 3 5 10:15 20:+5 )
```

The default is to select all pages to be printed.

SKIP number

SKIP
Skips a certain number of output lines, i.e., prints blank lines. SKIP will never print blank lines at the top of a page, so to skip lines at the top of a page at least one actual blank line must precede the SKIP directive. The default of SKIP is 5 lines.

SORTINDEX (parameters)

SORTINDEX
Index entries accumulated by INX directives can be sorted either alphabetically or by page number, then printed in a relatively flexible manner. The SORTINDEX directive allows you to specify the column that is to be considered the first significant column for alphabetical sorting, the number of leading blanks to print at the left of each index line, and the way to format the page number. The parameters, which may be given in any order, and listed as follows.

Key Letter	Default	Meaning
L	2	Left width of page number (field width of number)
M	0	Margin (left margin before index line)
P	0	Column (in index entry) to insert page number
R	2	Right width of page number, blanks printed after
S	1	Sorting option. If this is numeric, it is the first significant column for alphabetic sorting. If it is the letter "P", sorting is selected by page number.

In the absence of parameters, defaults are used.

SUBTITLE text

Enters the remainder of the directive line into the subtitle buffer. The subtitle buffer is used by the FORM directive.

TITLE text

Enters the remainder of the directive line into the main title buffer. The title buffer is used by the FORM directive.

UNDENT number

UNDENT

Undents the next line a certain number of spaces. The undent is sometimes known by the name "outdent" or "hanging indent". A line can never be undented past the leftmost column of the printer page, so a large number is adjusted to a smaller value. In the absence of a parameter, the default is to undent to the leftmost printable column.

WEOS

Write an end-of-section on the output file. This directive is useful for creating multiple section writeups under systems with utilities that manipulate multiple section files.

(6809/FLEX) This directive has no effect on the 6809/FLEX version of Prose.

Running Prose on the 6809/FLEX

After you have formatted the text as you desire, you "run" Prose and wait for the '*' prompt:

```
PROSE

*
```

Prose then accepts a command line of the following form:

```
        OUTPUT=INPUT1,INPUT2,INPUT3, ...
```

The input arguments are read and concatenated in the order listed.

If the output file and the "=" are omitted, the name of the last input file will be used with the extension ".DOC". Such a default output file will be placed in the default directory. The default extension for the output is ".DOC" and the default extension for the input is ".PRS".

If the output file already exists on the device, another question will be asked:

```
    Output file already exists; continue (Y/N)?
```

and a 'Y' or 'N' reply should be entered by the user.

We recommend as a general practice that you set up each Prose text without OUTPUT or FORM directives. Instead, keep these directives in another file that you will use as the first argument to the input line.

Extra utilities to use under FLEX

To enable the various way of listing the resulting "DOC" files from PROSE under the FLEX operating system, some extra utilities are available on the distribution diskette.

LISTAL

This utility should be used instead of the standard FLEX "LIST" utility when listing the "DOC" file on a DEC VT100 terminal, (specified output "VTR"); the LISTAL utility, unlike the LIST, will pass also all escape codes to the terminal. The LISTAL utility follows the same syntax as LIST.

MISSTYPE

This utility can be used to print the "DOC" file on a printer on a page by page base, pausing between each page to enable the operator to insert a new sheet. The routine is called in the following way:

```
        MISSTYPE filename
```

or

```
        MISSTYPE
```

In the latter way the filename will be prompted for. The default file extension is "DOC".

OVERPRT

This utility patches the FLEX printer spooler. In the normal operating (non patched) mode, the spooler adds a l.f. after each c.r. and disregards an eventually already present l.f. immediately after a c.r.. The patching sets the spooler in the "overprinting" mode: i.e. each character will be printed as it is read from the file, without adding or deleting characters. This feature thereby will enable the necessary overprinting (for underlining) for Prose DOC-files.

The calling sequence is:

```
       OVERPRT ON  .....  to enable overprinting (patch)
       OVERPRT OFF .....  to disable overprinting (unpatch)
       OVERPRT     .....  to report the overprinting status
```